Principios básicos de Javascript

Variables en JavaScript

1. Teoría

Una variable se declara con **let** o **const**. Antiguamente se utilizaba **var** pero **ya no se usa para declarar variables.**

1.1 Ciclo de vida de una variable

Una variable en JavaScript **nace** cuando se declara y se **destruye** cuando ya no es accesible (depende de su **ámbito o scope**).

- **Ámbito global** → accesible desde cualquier parte del programa (ej: variables declaradas fuera de funciones).
- **Ámbito local** → accesible solo dentro de la función o bloque donde se declararon.
- **Ámbito de bloque {}** → válido para `let` y `const`, no para `var`.



Ejemplo:

```
if (true) {
    var x = 10;   // existe fuera del bloque
    let y = 20;   // existe solo dentro del bloque
    const z = 30; // existe solo dentro del bloque
}

console.log(x); // ✔ funciona
// console.log(y); // ❌ error: y no está definida
// console.log(z); // ❌ error: z no está definida
```

1.2 Hoisting

JavaScript **"eleva" (hoisting)** las declaraciones de variables al inicio de su contexto.

- Con `var`, la declaración se eleva, pero **sin valor**. (No usamos **var**).
- Con `let` y `const`, también se eleva, pero permanecen en la **zona muerta temporal (TDZ)** hasta que se inicializan.

Ejemplo:

```
console.log(a); // undefined (var se eleva sin valor)
var a = 5;

// console.log(b); // ❌ Error: no se puede acceder antes de inicializar
let b = 10;
```

1.3 Inicialización y valores por defecto

- Si declaras una variable con `var` o `let` pero no la inicializas, tendrá el valor `undefined`.
- Una `const` debe **obligatoriamente inicializarse** en el momento de su declaración.

Ejemplo:

```
let nombre;
console.log(nombre); // undefined

const PI; // ❌ error: falta inicialización
```

1.4 Mutabilidad

- Las variables con `const` **no pueden ser reasignadas**, pero **sí pueden cambiar su contenido** si almacenan objetos o arrays (porque lo que no cambia es la referencia en memoria).

Ejemplo:

```
const numeros = [1, 2, 3];
numeros.push(4); // ✔ permitido, modificamos el array
console.log(numeros); // [1, 2, 3, 4]

// numeros = [5, 6]; // ❌ error: no se puede reasignar
```

1.5 Buenas prácticas extra

- Usar **const por defecto**, y solo `let` si sabes que la variable va a cambiar.
- Evitar usar `var` en proyectos modernos.
- Nombrar variables con claridad (no `x1`, `dato`, `cosas`).

---

2. Errores comunes con variables

1. **Olvidar `let` o `const`**
   Si escribes sin declarar:

```
ciudad = "Madrid"; // se crea como variable global implícita (mala práctica)
```

   Esto puede causar errores graves.

2. **Confundir reasignación con mutación**

```
const lista = [1, 2];

lista.push(3); // ✔ permitido

lista = [1, 2, 3]; // ❌ error
```

3. **Redeclarar con `let` o `const` en el mismo bloque**

```
let edad = 25;

// let edad = 30; // ❌ error
```

---

3. Ejercicios propuestos (nivel ampliado)

1. **Ámbitos de bloque**
   Declara una variable `x` con `var` dentro de un `if`.
   Declara otra variable `y` con `let` dentro del mismo `if`.
   Comprueba en consola cuál es accesible fuera del bloque y explica por qué.

2. **Hoisting práctico**
   Intenta imprimir en consola una variable antes de declararla con:
   - `var`
   - `let`
   - `const`

     Describe los resultados que observas.

3. **Mutabilidad con objetos**
   Declara una constante `persona` con propiedades `nombre` y `edad`.

   - Cambia la propiedad `edad` a un nuevo valor.

- Intenta reasignar `persona` a otro objeto.
  Explica el resultado.

4. **Buenas prácticas**
   Imagina que estás programando un carrito de compras:

   - Declara con `const` el nombre del producto y su precio.
   - Declara con `let` la cantidad (porque puede variar).
   - Calcula el total y muestra una frase en consola.

5. **Variables sin inicializar**
   Declara una variable con `let` sin asignarle valor.
   Imprime su valor y explica qué significa `undefined` en este contexto.

Tipos de Datos y Conversiones en JavaScript

1. Teoría

1.1 Tipos de datos primitivos

Son los valores más básicos, **inmutables** (no se pueden modificar directamente).

- **`string`** → cadenas de texto

```
let nombre = "Ana";

let apellidos = 'Lopez';
```

- **`number`** → números (enteros y decimales, positivo o negativo)

```
let edad = 25;

let precio = 19.99;
```

- **`boolean`** → valores lógicos (`true` o `false`)

```
let esMayorDeEdad = true;
```

- **`undefined`** → valor de una variable declarada pero sin inicializar

```
let dato;

console.log(dato); // undefined
```

- **`null`** → ausencia intencional de valor

```
let respuesta = null;
```

- **`symbol`** → valores únicos, usados en programación avanzada

```
let id = Symbol("id");
```

- **`bigint`** → enteros muy grandes

```
let numeroGrande = 1234567890123456789012345678901234567890n;
```

1.2 Tipos de datos de objeto

Son colecciones de datos o estructuras más complejas.

- **`Object`** → pares clave-valor

```
let persona = { nombre: "Ana", edad: 25 };
```

- **`Array`** → listas ordenadas (

```
let frutas = ["manzana", "pera", "plátano"];
```

- **`Function`** → funciones también son objetos

```
function saludar() { console.log("Hola!"); }
```

Otros tipos de estructuras de datos:

- Map / WeakMap: mapas más avanzados.
- Set / WeakSet: conjuntos de valores únicos.
- Typed Arrays: arrays especializados en datos binarios.

1.3 Conversiones de tipo

Conversión implícita (coerción de tipos)

JavaScript convierte automáticamente un tipo en otro cuando lo necesita.

```
console.log("5" + 3); // "53" → number se convierte en string
console.log("5" - 3); // 2 → string se convierte en number
```

Conversión explícita

El programador indica el cambio de tipo.

- **A número**

```
Number("123"); // 123
parseInt("123.45"); // 123
parseFloat("123.45"); // 123.45
```

- **A cadena**

```
String(123); // "123"
123..toString(); // "123"
```

- **A booleano**

```
Boolean(0); // false
Boolean("Hola"); // true
```

1.4 Valores "falsy" y "truthy"

En JavaScript, algunos valores se consideran **falsos** al evaluarse en un contexto booleano.

- **Falsy**: `0`, `""`, `null`, `undefined`, `NaN`, `false`
- **Truthy**: cualquier otro valor

Ejemplo:

```
if ("") {
  console.log("Se ejecuta");
} else {
  console.log("No se ejecuta");
}
```

---

2. Ejercicios Propuestos

1. Declara una variable `nombre` (string), otra `edad` (number) y otra `esEstudiante` (boolean). Imprime los valores en consola.

2. Declara una variable sin valor e imprímela. Luego asígnale `null`. Explica la diferencia entre `undefined` y `null`.

3. Crea un array con tres frutas y muestra la primera y la última en consola.

4. Declara un objeto `coche` con propiedades `marca`, `modelo` y `año`. Imprime sus valores en consola.

5. Usa coerción implícita: suma un número y una cadena. ¿Qué resultado obtienes?

6. Convierte la cadena `"123.45"` en número con `parseInt` y con `parseFloat`. ¿Qué diferencias hay?

7. Convierte los siguientes valores a booleanos con `Boolean()`: `0`, `"0"`, `""`, `"hola"`, `null`.

⚠️ Errores Típicos con Conversiones en JavaScript

1. Confundir `undefined` y `null`

- `undefined` → variable declarada pero sin valor.
- `null` → ausencia de valor asignada intencionalmente.

```
let a;

let b = null;


console.log(a); // undefined

console.log(b); // null
```

---

2. Creer que `"false"` es falso

Cualquier cadena **no vacía** es `truthy`, incluso `"false"`.

```
Boolean("false"); // true ✔
```

---

3. Olvidar que `+` concatena si hay strings

Cuando se combina un string con un número, el resultado siempre es string.

```
console.log("5" + 2); // "52", no 7
```

---

4. Pensar que `parseInt` sirve para decimales

`parseInt` elimina la parte decimal, lo que genera confusión.

```
parseInt("12.7"); // 12 ❌ pierde la parte decimal

parseFloat("12.7"); // 12.7 ✔
```

---

5. Usar `Number()` sin comprobar cadenas inválidas

Si la cadena no es numérica, el resultado será `NaN` (*Not a Number*).

```
Number("abc"); // NaN
```

---

6. No diferenciar `==` de `===`

- `==` convierte tipos antes de comparar (comparación flexible).
- `===` compara tipo y valor (comparación estricta).

```
5 == "5"; // true ✔

5 === "5"; // false ✔
```

---

7. Pensar que `NaN` es igual a sí mismo

`NaN` es el único valor en JS que **no es igual a sí mismo**.

```
console.log(NaN === NaN); // false
```

```
isNaN(NaN); // true ✔
```

---

8. Confundir cadena vacía con `null` o `undefined`

- `""` (cadena vacía) → `false` al convertir a booleano.
- `null` y `undefined` → también `false`, pero **no son lo mismo**.

```
Boolean(""); // false

Boolean(null); // false

Boolean(undefined); // false
```

---

9. Esperar que arrays vacíos sean falsos

Un array vacío (`[]`) es **truthy**, aunque no tenga elementos.

```
Boolean([]); // true ✔
```

---

10. No entender la precisión de `Number`

Los números muy grandes pierden precisión si no se usa `BigInt`.

```
let n = 1234567890123456789;

console.log(n); // 1234567890123456788 ❌ (pierde precisión)

let big = 1234567890123456789n;

console.log(big); // 1234567890123456789n ✔
```